CS-462 – Programming Language Foundations
Lecture #15: Type Inference and Let-Polymorphism

EPFL

1 Polymorphism in OCaml

In System F, programmers must write type abstractions, type applications, and type annotations explicitly. For example, to use the polymorphic twice function at type **int**, we must write twice [**int**] (λn : **int**. $n + 1$) 7. As we saw last time, it is undecidable whether an untyped term can be annotated to obtain a well-typed System F term (Wells 1994), so a compiler cannot infer these annotations in general.

Yet in languages like OCaml, programmers do not have to write types at all, and they still get polymorphism. For example, we can write

```
let twice f x = f (f x)
```

and OCaml will infer that its type is

```
('a -> 'a) -> 'a -> 'a
```

which corresponds to the System F type $\forall X.\ (X \to X) \to X \to X$. We can then write `twice (fun n -> n + 1) 7` or `twice not true`, and OCaml will automatically instantiate the type variable at **int** or at **bool**.

OCaml can do this because its polymorphism is more restricted than System F's. In this lecture, we will first see how to infer types for the simply-typed λ-calculus, by generating *constraints* and solving them using *unification*. Then we will extend this approach to *let-polymorphism*, the restricted form of polymorphism used by OCaml and other languages in the ML family.

2 Type Inference

In the simply-typed λ-calculus, every function argument has a type annotation $\lambda x : \tau.\, e$, which is used by the typing rule for abstractions. Suppose we didn't want to write these annotations. Can we still type check the program, by *inferring* (or *reconstructing*) the missing types? For the simply-typed λ-calculus, the answer is yes.

Let's consider an example to see how type inference could work:

$$\lambda a.\, \lambda b.\, \lambda c.\, \mathsf{if}\ a\ (b + 1)\ \mathsf{then}\ b\ \mathsf{else}\ c$$

Since the variable b is used in an addition, the type of b must be **int**. The variable a is applied to the expression $b + 1$, so it must be a function, and its argument type must be the type of $b + 1$, namely **int**. The result of the application $a\ (b + 1)$ is used as the guard of a conditional, so the result type of a must be **bool**. Finally, both branches of a conditional must have the same type, so the type of c must be the same as the type of b, namely **int**. So we can reconstruct the types:

$$\lambda a : \mathbf{int} \to \mathbf{bool}.\, \lambda b : \mathbf{int}.\, \lambda c : \mathbf{int}.\, \mathsf{if}\ a\ (b + 1)\ \mathsf{then}\ b\ \mathsf{else}\ c$$

2.1 Constraint-Based Typing

We now present an algorithm that, given a typing context Γ and an expression e, produces a type together with a set of *constraints*—equations between types—that must be satisfied for e to be well-typed. We extend the types of the simply-typed λ-calculus with *type variables* $X, Y, \ldots$, which are placeholders for unknown types, and we allow functions both with and without type annotations:

$$e ::= x \mid \lambda x{:}\tau.\, e \mid \lambda x.\, e \mid e_1\ e_2 \mid () \mid n \mid e_1 \oplus e_2 \mid \mathsf{true} \mid \mathsf{false} \mid \mathsf{if}\ e_1\ \mathsf{then}\ e_2\ \mathsf{else}\ e_3$$
$$\tau ::= \mathbf{unit} \mid \mathbf{int} \mid \mathbf{bool} \mid \tau_1 \to \tau_2 \mid X$$

Note that these type variables are different from the type variables of System F: they are not bound by any $\forall$, but stand for particular types that we have not figured out yet.

The new typing judgment has the form $\Gamma \vdash e : \tau \mid C$. Intuitively, it means that expression e has type τ in context Γ, provided that every constraint in the set C is satisfied. We define it with the following inference rules. When read from bottom to top, the rules give a procedure that, given Γ and e, calculates τ and C.

$$\textsc{CT-Var}\ \frac{\Gamma(x) = \tau}{\Gamma \vdash x : \tau \mid \emptyset} \qquad \textsc{CT-Int}\ \frac{}{\Gamma \vdash n : \mathbf{int} \mid \emptyset} \qquad \textsc{CT-True}\ \frac{}{\Gamma \vdash \mathsf{true} : \mathbf{bool} \mid \emptyset}$$

$$\textsc{CT-Arith}\ \frac{\Gamma \vdash e_1 : \tau_1 \mid C_1 \quad \Gamma \vdash e_2 : \tau_2 \mid C_2}{\Gamma \vdash e_1 \oplus e_2 : \mathbf{int} \mid C_1 \cup C_2 \cup \{\tau_1 = \mathbf{int}, \tau_2 = \mathbf{int}\}}$$

$$\textsc{CT-If}\ \frac{\Gamma \vdash e_1 : \tau_1 \mid C_1 \quad \Gamma \vdash e_2 : \tau_2 \mid C_2 \quad \Gamma \vdash e_3 : \tau_3 \mid C_3}{\Gamma \vdash \mathsf{if}\ e_1\ \mathsf{then}\ e_2\ \mathsf{else}\ e_3 : \tau_2 \mid C_1 \cup C_2 \cup C_3 \cup \{\tau_1 = \mathbf{bool}, \tau_2 = \tau_3\}}$$

$$\textsc{CT-AbsAnn}\ \frac{\Gamma, x : \tau_1 \vdash e : \tau_2 \mid C}{\Gamma \vdash \lambda x{:}\tau_1.\, e : \tau_1 \to \tau_2 \mid C} \qquad \textsc{CT-Abs}\ \frac{\Gamma, x : X \vdash e : \tau \mid C}{\Gamma \vdash \lambda x.\, e : X \to \tau \mid C}\ X \text{ fresh}$$

$$\textsc{CT-App}\ \frac{\Gamma \vdash e_1 : \tau_1 \mid C_1 \quad \Gamma \vdash e_2 : \tau_2 \mid C_2}{\Gamma \vdash e_1\ e_2 : X \mid C_1 \cup C_2 \cup \{\tau_1 = \tau_2 \to X\}}\ X \text{ fresh}$$

The rules for () and false are similar to CT-True. In CT-AbsAnn, we use the annotation as the type of the argument. In CT-Abs, we do not know the type of the argument, so we invent a fresh type variable X for it. In CT-App, we do not know the result type of the application, so we invent a fresh type variable X for it, and add a constraint saying that e_1 must be a function from the type of e_2 to X. Here "fresh" means that X must not appear anywhere else in the derivation.

For example, consider our expression again. Rule CT-Abs gives a, b, and c the fresh types X_a, X_b, and X_c. Then for the subexpression $b + 1$, CT-Arith gives the type **int** and the constraints $\{X_b = \mathbf{int}, \mathbf{int} = \mathbf{int}\}$. For the application $a\ (b + 1)$, CT-App gives a fresh type Y and adds the constraint $X_a = \mathbf{int} \to Y$. Finally, for the conditional, CT-If gives the type X_b and adds the constraints $Y = \mathbf{bool}$ and $X_b = X_c$. So the whole expression has type

$$X_a \to X_b \to X_c \to X_b$$

subject to the constraints

$$C = \{X_b = \textbf{int},\ \textbf{int} = \textbf{int},\ X_a = \textbf{int} \to Y,\ Y = \textbf{bool},\ X_b = X_c\}.$$

2.2 Unification

What does it mean for a set of constraints to be satisfied? To answer this question, we define *type substitutions*. A type substitution σ is a finite map from type variables to types. For example, we write $[X \mapsto \textbf{int}, Y \mapsto \textbf{int} \to \textbf{int}]$ for the substitution that maps X to **int** and Y to **int** $\to$ **int**. Applying a substitution to a type replaces its type variables simultaneously:

$$\sigma(X) = \begin{cases} \tau & \text{if } X \mapsto \tau \in \sigma \\ X & \text{if } X \text{ is not in the domain of } \sigma \end{cases}$$
$$\sigma(\tau_1 \to \tau_2) = \sigma(\tau_1) \to \sigma(\tau_2)$$

and $\sigma(\tau) = \tau$ for the base types **unit**, **int**, and **bool**. Since no construct binds these type variables, we do not need to worry about capture. We write $\sigma_1 \circ \sigma_2$ for the composition of two substitutions, with $(\sigma_1 \circ \sigma_2)(\tau) = \sigma_1(\sigma_2(\tau))$.

A substitution σ *unifies* a constraint $\tau_1 = \tau_2$ if $\sigma(\tau_1) = \sigma(\tau_2)$, and it *satisfies* (or *unifies*) a set of constraints C if it unifies every constraint in C. For example, $[X \mapsto \textbf{int}, Y \mapsto \textbf{int} \to \textbf{int}]$ unifies the constraint $X \to (X \to \textbf{int}) = \textbf{int} \to Y$, since applying it to both sides gives **int** $\to$ (**int** $\to$ **int**).

If $\Gamma \vdash e : \tau \mid C$ and σ satisfies C, then e has type $\sigma(\tau)$ in $\sigma(\Gamma)$. If no substitution satisfies C, then e is not typable. So to infer a type for e, we need to find a substitution that satisfies C. The following *unification* algorithm does exactly that:

$$\begin{aligned}
\mathit{unify}(\emptyset) &= [\,] \qquad \text{(the empty substitution)} \\
\mathit{unify}(\{\tau_1 = \tau_2\} \cup C) &= \text{if } \tau_1 = \tau_2 \text{ then} \\
&\qquad \mathit{unify}(C) \\
&\quad \text{else if } \tau_1 = X \text{ and } X \notin \mathit{ftv}(\tau_2) \text{ then} \\
&\qquad \mathit{unify}(C\{\tau_2/X\}) \circ [X \mapsto \tau_2] \\
&\quad \text{else if } \tau_2 = X \text{ and } X \notin \mathit{ftv}(\tau_1) \text{ then} \\
&\qquad \mathit{unify}(C\{\tau_1/X\}) \circ [X \mapsto \tau_1] \\
&\quad \text{else if } \tau_1 = \tau_1' \to \tau_1'' \text{ and } \tau_2 = \tau_2' \to \tau_2'' \text{ then} \\
&\qquad \mathit{unify}(C \cup \{\tau_1' = \tau_2',\ \tau_1'' = \tau_2''\}) \\
&\quad \text{else} \\
&\qquad \mathit{fail}
\end{aligned}$$

Here $\mathit{ftv}(\tau)$ is the set of type variables that occur in τ. The algorithm fails when it is asked to unify two different base types, or a base type with a function type.

For example, here is how *unify* solves the constraints C from our example. The constraint $X_b =$ **int** produces the substitution $[X_b \mapsto \textbf{int}]$, and turns the constraint $X_b = X_c$ into **int** $= X_c$. The constraint **int** $=$ **int** is already satisfied. The constraint $X_a =$ **int** $\to Y$ produces $[X_a \mapsto \textbf{int} \to Y]$, then $Y =$ **bool** produces $[Y \mapsto \textbf{bool}]$, and finally **int** $= X_c$ produces $[X_c \mapsto \textbf{int}]$. Applying the composition of these substitutions to $X_a \to X_b \to X_c \to X_b$ gives

$$(\textbf{int} \to \textbf{bool}) \to \textbf{int} \to \textbf{int} \to \textbf{int},$$

which is what we found informally.

The unification algorithm, due to Robinson (1965), was applied to type inference by Hindley (1969) and Milner (1978). It has the following properties.

Theorem (Unification)**.** *The algorithm $unify(C)$ always terminates. It succeeds if and only if C has a solution, and in that case $unify(C)$ is a* most general *solution: for every solution σ' of C, there is a substitution σ'' such that $\sigma' = \sigma'' \circ unify(C)$.*

The last property means that the type that we infer is the most general one: every other type that the expression could be given is an instance of it. For example, for $\lambda x.\, x$, we generate the type $X \to X$ with no constraints, so *unify* returns the empty substitution, and the inferred type is $X \to X$. Every other type for $\lambda x.\, x$, such as **int** $\to$ **int**, is an instance of $X \to X$.

2.3 The Occurs Check

The check that $X \notin ftv(\tau)$ before mapping X to τ is called the *occurs check*. It ensures that the algorithm does not produce a cyclic substitution such as $[X \mapsto X \to Y]$, which would make sense only with infinite types. For example, for the self-application $\lambda x.\, x\ x$, rule CT-Abs gives x the type X, and CT-App generates the constraint $X = X \to Y$. The occurs check makes *unify* fail, so $\lambda x.\, x\ x$ is not typable, just as in the simply-typed λ-calculus. This is the source of OCaml's error message "The type variable `'a` occurs inside `'a -> 'b`" for `fun x -> x x`.

3 Let-Polymorphism

Unification finds the most general type of an expression, which may contain type variables that are not constrained at all. For example, $\lambda x.\, x$ has type $X \to X$. It is natural to want to use such a function at several different types:

$$\mathsf{let\ id} = \lambda x.\, x\ \mathsf{in\ if\ id\ true\ then\ id}\ 1\ \mathsf{else}\ 0$$

(Here we treat $\mathsf{let}\ x = e_1\ \mathsf{in}\ e_2$ as an abbreviation for $(\lambda x.\, e_2)\ e_1$, as in the simply-typed λ-calculus.) With the rules so far, id has the single type $X \to X$, so its two uses generate the constraints $X =$ **bool** and $X =$ **int**, which cannot both be satisfied.

The idea of *let-polymorphism* is that when we define id with a let, we know its whole definition. So we can *generalize* its type to $\forall X.\ X \to X$, and then *instantiate* it with a fresh type variable each time it is used. To keep inference decidable, we only allow $\forall$ at the outside of a type. A *type scheme* is a type with some type variables universally quantified at the outside:

$$s ::= \tau \mid \forall X.\, s$$

and typing contexts now map variables to type schemes. The types τ themselves contain no $\forall$s, so we call them *monotypes*.

3.1 Typing Rules

Here are the typing rules for variables, abstractions, and let (the other rules are as in the simply-typed λ-calculus):

$$\text{T-Var}\ \frac{\Gamma(x) = \forall X_1 \ldots X_n.\ \tau}{\Gamma \vdash x : \tau\{\tau_1/X_1, \ldots, \tau_n/X_n\}} \qquad \text{T-Abs}\ \frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \lambda x.\, e : \tau_1 \to \tau_2}$$

$$\text{T-Let}\ \frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma, x : gen(\Gamma, \tau_1) \vdash e_2 : \tau_2}{\Gamma \vdash \mathsf{let}\ x = e_1\ \mathsf{in}\ e_2 : \tau_2}$$

Here $gen(\Gamma, \tau) = \forall X_1 \ldots X_n.\ \tau$, where $X_1, \ldots, X_n$ are the type variables that are free in τ but not free in Γ. We cannot generalize over type variables that appear in Γ, because they stand for the types of variables that are still in scope, which are not yet known. Rule T-Var instantiates the quantified type variables of a type scheme with arbitrary monotypes, and rule T-Abs gives the argument of a function a monotype.

With these rules, id gets the type scheme $\forall X.\ X \to X$, and the two uses of id instantiate X with **bool** and with **int**. So the whole expression has type **int**.

Note that let is no longer just an abbreviation:

$$(\lambda \mathsf{id}.\ \mathsf{if\ id\ true\ then\ id\ 1\ else\ 0})\ (\lambda x.\, x)$$

is *not* typable, because the λ-bound variable id must have a single monotype. This program is well-typed in System F, but only with the type $(\forall X.\ X \to X) \to$ **int**, which has a $\forall$ to the left of an arrow. The restriction that $\forall$ may only appear at the outside of a type scheme is called *prenex polymorphism*, and it is what makes type inference decidable. In return, every typable expression has a *principal type scheme*, of which all its other types are instances.

3.2 Inference

To infer types with let-polymorphism, we combine constraint generation with unification. To infer the type of $\mathsf{let}\ x = e_1\ \mathsf{in}\ e_2$ in Γ:

1. Generate constraints for e_1, so that $\Gamma \vdash e_1 : \tau_1 \mid C_1$.
2. Solve them right away: let $\sigma = \mathit{unify}(C_1)$.
3. Generalize: give x the type scheme $gen(\sigma(\Gamma), \sigma(\tau_1))$.
4. Continue by inferring the type of e_2 in the context $\sigma(\Gamma)$ extended with this type scheme for x.

And at each use of a variable whose type scheme is $\forall X_1 \ldots X_n.\ \tau$, we replace $X_1, \ldots, X_n$ with fresh type variables. We have to solve the constraints for e_1 before generalizing, since otherwise we would not know which type variables are really unconstrained.

This is essentially the Hindley–Milner type inference algorithm (often called *Algorithm W*), which is the core of type inference in OCaml, Haskell, and other languages. It is fast in practice, although in the worst case, nested lets can produce types whose size is exponential in the size of the program.

3.3 The Value Restriction

In a language with references, generalizing every let is unsound. Consider this OCaml program:

```
let r = ref (fun x -> x) in
r := (fun n -> n + 1);
(!r) true
```

If we generalized the type of `r` to $\forall X.\ (X \to X)$ **ref**, then the assignment could use it at **int** and the dereference at **bool**, and the program would apply a function on integers to a Boolean. OCaml avoids this problem with the *value restriction*: it only generalizes the type of let $x = e_1$ in e_2 when e_1 is a syntactic value, such as a function. Otherwise, the type variables remain *weak*, and the first use of x determines them. So in the program above, the assignment fixes the type of `r` to (**int** $\to$ **int**) **ref**, and the application to `true` is rejected.

3.4 Weak Type Variables

OCaml prints weak type variables as `'_weak1`, `'_weak2`, and so on. Whereas `'a` means *for all* types, `'_weak1` means *one particular* type that has not been determined yet. In the toplevel:

```
# let r = ref (fun x -> x);;
val r : ('_weak1 -> '_weak1) ref
# r := (fun n -> n + 1);;
- : unit = ()
# r;;
- : (int -> int) ref
# (!r) true;;
Error: This expression has type bool but an expression was expected of type int
```

In terms of our algorithm, the value restriction solves the constraints for e_1 as usual, but skips gen. A weak type variable is therefore just an unsolved type variable that is free in the typing context. Later constraints can still solve it, as for any other type variable, but since it stands for a single type, it can only be solved once. This is why the restriction is sound: a reference can only ever hold values of one type.

A common surprise is that the value restriction also applies when nothing is mutable, because an application is not a value:

```
# let ids = List.map (fun x -> x);;
val ids : '_weak1 list -> '_weak1 list
```

The fix is to η-expand the definition, which turns it into a function, and so a value:

```
# let ids l = List.map (fun x -> x) l;;
val ids : 'a list -> 'a list
```

A weak type variable cannot escape a compilation unit: if nothing in a file determines it, the compiler reports that the type contains type variables that cannot be generalized. Finally, OCaml actually uses a *relaxed* value restriction (Garrigue, 2004), which also generalizes type variables that appear only in covariant positions. For example, `let x = List.rev []` has type `'a list`, since `'a` does not appear under `ref` or to the left of an arrow.

4 Summary

System F provides fully general polymorphism, but programs must be annotated with types, since type inference is undecidable. For the simply-typed λ-calculus, we can infer types by generating constraints and solving them with unification. Let-polymorphism extends this approach to a restricted form of polymorphism, with $\forall$ only at the outside of type schemes, which are generalized at let and instantiated at each use. Inference for let-polymorphism is decidable and finds principal types, which is why OCaml can infer polymorphic types for us.